

Or—if it makes sense for your scenario, and if threading considerations make it even possible—then another option is to revoke the handler after the recipient is done with the event, or in the recipient's destructor. See [Revoke a registered delegate](#).

This is how we're registering the handler.

C++/WinRT

```
event_source.Event([&](auto&& ...)  
{  
    std::wcout << m_value.c_str() << std::endl;  
});
```

The lambda automatically captures any local variables by reference. So, for this example, we could equivalently have written this.

C++/WinRT

```
event_source.Event([this](auto&& ...)  
{  
    std::wcout << m_value.c_str() << std::endl;  
});
```

In both cases, we're just capturing the raw *this* pointer. And that has no effect on reference-counting, so nothing is preventing the current object from being destroyed.

The solution

The solution is to capture a strong reference (or, as we'll see, a weak reference if that's more appropriate). A strong reference *does* increment the reference count, and it *does* keep the current object alive. You just declare a capture variable (called `strong_this` in this example), and initialize it with a call to `implements::get_strong`, which retrieves a strong reference to our *this* pointer.

Important

Because `get_strong` is a member function of the `wintrt::implements` struct template, you can call it only from a class that directly or indirectly derives from `wintrt::implements`, such as a C++/WinRT class. For more info about deriving from `wintrt::implements`, and examples, see [Author APIs with C++/WinRT](#).

C++/WinRT

```
event_source.Event([this, strong_this { get_strong() }](auto&& ...)  
{  
    std::wcout << m_value.c_str() << std::endl;  
});
```

You can even omit the automatic capture of the current object, and access the data member through the capture variable instead of via the implicit *this*.

C++/WinRT

```
event_source.Event([strong_this { get_strong() }](auto&& ...)  
{  
    std::wcout << strong_this->m_value.c_str() << std::endl;  
});
```

If a strong reference isn't appropriate, then you can instead call [implements::get_weak](#) to retrieve a weak reference to *this*. A weak reference *does not* keep the current object alive. So, just confirm that you can still retrieve a strong reference from the weak reference before accessing members.

C++/WinRT

```
event_source.Event([weak_this{ get_weak() }](auto&& ...)  
{  
    if (auto strong_this{ weak_this.get() })  
    {  
        std::wcout << strong_this->m_value.c_str() << std::endl;  
    }  
});
```

If you capture a raw pointer, then you'll need to make sure you keep the pointed-to object alive.

If you use a member function as a delegate

As well as lambda functions, these principles also apply to using a member function as your delegate. The syntax is different, so let's look at some code. First, here's the potentially unsafe member function event handler, using a raw *this* pointer.

C++/WinRT

```
struct EventRecipient : winrt::implements<EventRecipient, IInspectable>  
{  
    winrt::hstring m_value{ L"Hello, World!" };
```

```

void Register(EventSource& event_source)
{
    event_source.Event({ this, &EventRecipient::OnEvent });
}

void OnEvent(IInspectable const& /* sender */, int /* args */)
{
    std::wcout << m_value.c_str() << std::endl;
}
};

```

This is the standard, conventional way to refer to an object and its member function. To make this safe, you can—as of version 10.0.17763.0 (Windows 10, version 1809) of the Windows SDK—establish a strong or a weak reference at the point where the handler is registered. At that point, the event recipient object is known to be still alive.

For a strong reference, just call [get_strong](#) in place of the raw *this* pointer. C++/WinRT ensures that the resulting delegate holds a strong reference to the current object.

C++/WinRT

```

event_source.Event({ get_strong(), &EventRecipient::OnEvent });

```

Capturing a strong reference means that your object will become eligible for destruction only after the handler has been unregistered and all outstanding callbacks have returned. However, that guarantee is valid only at the time the event is raised. If your event handler is asynchronous, then you'll have to give your coroutine a strong reference to the class instance before the first suspension point (for details, and code, see the [Safely accessing the *this* pointer in a class-member coroutine](#) section earlier in this topic). But that creates a circular reference between the event source and your object, so you need to explicitly break that by revoking your event.

For a weak reference, call [get_weak](#). C++/WinRT ensures that the resulting delegate holds a weak reference. At the last minute, and behind the scenes, the delegate attempts to resolve the weak reference to a strong one, and only calls the member function if it's successful.

C++/WinRT

```

event_source.Event({ get_weak(), &EventRecipient::OnEvent });

```

If the delegate *does* call your member function, then C++/WinRT will keep your object alive until your handler returns. However, if your handler is asynchronous, then it returns at suspension points, and so you'll have to give your coroutine a strong reference to the

class instance before the first suspension point. Again, for more info, see [Safely accessing the *this* pointer in a class-member coroutine](#) section earlier in this topic.

If the member function doesn't belong to a Windows Runtime type

When the [get_strong](#) method isn't available to you (your type isn't a Windows Runtime type), you can use the technique shown in the code example below. Here, a regular C++ class (named **ConsoleNetworkWatcher**) is shown handling the [NetworkInformation.NetworkStatusChanged](#) event.

C++/WinRT

```
#include <winrt/Windows.Networking.Connectivity.h>
using namespace winrt;
using namespace Windows::Networking::Connectivity;

class ConsoleNetworkWatcher
{
    /* any constructor, and instance methods, here*/

    static void Initialize(std::shared_ptr<ConsoleNetworkWatcher> instance)
    {
        auto weakPointer{ std::weak_ptr{ instance } };

        instance->m_statusChangedRevoker =
            NetworkInformation::NetworkStatusChanged(winrt::auto_revoke,
                [weakPointer](winrt::Windows::Foundation::IIInspectable
const& sender)
                {
                    auto sharedPointer{ weakPointer.lock() };

                    if (sharedPointer)
                    {
                        sharedPointer->NetworkStatusChanged(sender);
                    }
                });
    }

    void NetworkStatusChanged(winrt::Windows::Foundation::IIInspectable
const& sender){/* handle event here */};

private:
    NetworkInformation::NetworkStatusChanged_revoker m_statusChangedRevoker;
};
```

A weak reference example using SwapChainPanel::CompositionScaleChanged

In this code example, we use the [SwapChainPanel::CompositionScaleChanged](#) event by way of another illustration of weak references. The code registers an event handler using a lambda that captures a weak reference to the recipient.

C++/WinRT

```
wintrt::Windows::UI::Xaml::Controls::SwapChainPanel m_swapChainPanel;
winrt::event_token m_compositionScaleChangedEventToken;

void RegisterEventHandler()
{
    m_compositionScaleChangedEventToken =
    m_swapChainPanel.CompositionScaleChanged([weak_this{ get_weak() }]
        (Windows::UI::Xaml::Controls::SwapChainPanel const& sender,
         Windows::Foundation::IInspectable const& object)
        {
            if (auto strong_this{ weak_this.get() })
            {
                strong_this->OnCompositionScaleChanged(sender, object);
            }
        });
}

void OnCompositionScaleChanged(Windows::UI::Xaml::Controls::SwapChainPanel
const& sender,
    Windows::Foundation::IInspectable const& object)
{
    // Here, we know that the "this" object is valid.
}
```

In the lambda capture clause, a temporary variable is created, representing a weak reference to *this*. In the body of the lambda, if a strong reference to *this* can be obtained, then the **OnCompositionScaleChanged** function is called. That way, inside **OnCompositionScaleChanged**, *this* can safely be used.

Weak references in C++/WinRT

Above, we saw weak references being used. In general, they're good for breaking cyclic references. For example, for the native implementation of the XAML-based UI framework—because of the historical design of the framework—the weak reference mechanism in C++/WinRT is necessary to handle cyclic references. Outside of XAML, though, you likely won't need to use weak references (not that there's anything inherently XAML-specific about them). Rather you should, more often than not, be able to design your own C++/WinRT APIs in such a way as to avoid the need for cyclic references and weak references.

For any given type that you declare, it's not immediately obvious to C++/WinRT whether or when weak references are needed. So, C++/WinRT provides weak reference support automatically on the struct template [winrt::implements](#), from which your own C++/WinRT types directly or indirectly derive. It's pay-for-play, in that it doesn't cost you anything unless your object is actually queried for [IWeakReferenceSource](#). And you can choose explicitly to [opt out of that support](#).

Code examples

The [winrt::weak_ref](#) struct template is one option for getting a weak reference to a class instance.

C++/WinRT

```
Class c;  
winrt::weak_ref<Class> weak{ c };
```

Or, you can use the [winrt::make_weak](#) helper function.

C++/WinRT

```
Class c;  
auto weak = winrt::make_weak(c);
```

Creating a weak reference doesn't affect the reference count on the object itself; it just causes a control block to be allocated. That control block takes care of implementing the weak reference semantics. You can then try to promote the weak reference to a strong reference and, if successful, use it.

C++/WinRT

```
if (Class strong = weak.get())  
{  
    // use strong, for example strong.DoWork();  
}
```

Provided that some other strong reference still exists, the [weak_ref::get](#) call increments the reference count and returns the strong reference to the caller.

Opting out of weak reference support

Weak reference support is automatic. But you can choose explicitly to opt out of that support by passing the [winrt::no_weak_ref](#) marker struct as a template argument to

your base class.

If you derive directly from `winrt::implements`.

C++/WinRT

```
struct MyImplementation: implements<MyImplementation, IStringable,  
no_weak_ref>  
{  
    ...  
}
```

If you're authoring a runtime class.

C++/WinRT

```
struct MyRuntimeClass: MyRuntimeClassT<MyRuntimeClass, no_weak_ref>  
{  
    ...  
}
```

It doesn't matter where in the variadic parameter pack the marker struct appears. If you request a weak reference for an opted-out type, then the compiler will help you out with *"This is only for weak ref support"*.

Important APIs

- [implements::get_weak](#) function
- [winrt::make_weak](#) function template
- [winrt::no_weak_ref](#) marker struct
- [winrt::weak_ref](#) struct template

Agile objects in C++/WinRT

Article • 10/20/2022

In the vast majority of cases, an instance of a Windows Runtime class can be accessed from any thread (just like most standard C++ objects can). Such a Windows Runtime class is *agile*. Only a small number of Windows Runtime classes that ship with Windows are non-agile, but when you consume them you need to take into consideration their threading model and marshaling behavior (marshaling is passing data across an apartment boundary). It's a good default for every Windows Runtime object to be agile, so your own C++/WinRT types are agile by default.

But you can opt out. You might have a compelling reason to require an object of your type to reside, for example, in a given single-threaded apartment. This typically has to do with reentrancy requirements. But increasingly, even user interface (UI) APIs offer agile objects. In general, agility is the simplest and most performant option. Also, when you implement an activation factory, it must be agile even if your corresponding runtime class isn't.

ⓘ Note

The Windows Runtime is based on COM. In COM terms, an agile class is registered with `ThreadingModel = Both`. For more info about COM threading models, and apartments, see [Understanding and Using COM Threading Models](#).

Code examples

Let's use an example implementation of a runtime class to show how C++/WinRT supports agility.

C++/WinRT

```
#include <winrt/Windows.Foundation.h>

using namespace winrt;
using namespace Windows::Foundation;

struct MyType : winrt::implements<MyType, IStringable>
{
    winrt::hstring ToString(){ ... }
};
```


Because we haven't opted out, this implementation is agile. The `winrt::implements` base struct implements `IAgileObject` and `IMarshal`. The `IMarshal` implementation uses `CoCreateFreeThreadedMarshaler` to do the right thing for legacy code that doesn't know about `IAgileObject`.

This code checks an object for agility. The call to `IUnknown::as` throws an exception if `myimpl` is not agile.

C++/WinRT

```
winrt::com_ptr<MyType> myimpl{ winrt::make_self<MyType>() };
winrt::com_ptr<IAgileObject> iagileobject{ myimpl.as<IAgileObject>() };
```

Rather than handle an exception, you can call `IUnknown::try_as` instead.

C++/WinRT

```
winrt::com_ptr<IAgileObject> iagileobject{ myimpl.try_as<IAgileObject>() };
if (iagileobject) { /* myimpl is agile. */ }
```

`IAgileObject` has no methods of its own, so you can't do much with it. This next variant, then, is more typical.

C++/WinRT

```
if (myimpl.try_as<IAgileObject>()) { /* myimpl is agile. */ }
```

`IAgileObject` is a *marker interface*. The mere success or failure of querying for `IAgileObject` is the extent of the information and utility you get from it.

Opt out of agile object support

You can choose explicitly to opt out of agile object support by passing the `winrt::non_agile` marker struct as a template argument to your base class.

If you derive directly from `winrt::implements`.

C++/WinRT

```
struct MyImplementation: implements<MyImplementation, IStringable,
winrt::non_agile>
{
    ...
}
```

If you're authoring a runtime class.

C++/WinRT

```
struct MyRuntimeClass: MyRuntimeClassT<MyRuntimeClass, winrt::non_agile>
{
    ...
}
```

It doesn't matter where in the variadic parameter pack the marker struct appears.

Whether or not you opt out of agility, you can implement **IMarshal** yourself. For example, you can use the **winrt::non_agile** marker to avoid the default agility implementation, and implement **IMarshal** yourself—perhaps to support marshal-by-value semantics.

Agile references (winrt::agile_ref)

If you're consuming an object that isn't agile, but you need to pass it around in some potentially agile context, then one option is to use the **winrt::agile_ref** struct template to get an agile reference to an instance of a non-agile type, or to an interface of a non-agile object.

C++/WinRT

```
NonAgileType nonagile_obj;
winrt::agile_ref<NonAgileType> agile{ nonagile_obj };
```

Or, you can use the **winrt::make_agile** helper function.

C++/WinRT

```
NonAgileType nonagile_obj;
auto agile{ winrt::make_agile(nonagile_obj) };
```

In either case, **agile** may now be freely passed to a thread in a different apartment, and used there.

C++/WinRT

```
co_await resume_background();
NonAgileType nonagile_obj_again{ agile.get() };
winrt::hstring message{ nonagile_obj_again.Message() };
```

The [agile_ref::get](#) call returns a proxy that may safely be used within the thread context in which **get** is called.

Important APIs

- [IAgileObject interface](#)
- [IMarshal interface](#)
- [winrt::agile_ref struct template](#)
- [winrt::implements struct template](#)
- [winrt::make_agile function template](#)
- [winrt::non_agile marker struct](#)
- [winrt::Windows::Foundation::IUnknown::as function](#)
- [winrt::Windows::Foundation::IUnknown::try_as function](#)

Related topics

- [Understanding and Using COM Threading Models](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Diagnosing direct allocations

Article • 10/20/2022

As explained in [Author APIs with C++/WinRT](#), when you create an object of implementation type, you should use the `wintrt::make` family of helpers to do so. This topic goes in-depth on a C++/WinRT 2.0 feature that helps you to diagnose the mistake of directly allocating an object of implementation type on the stack.

Such mistakes can turn into mysterious crashes or corruptions that are difficult and time-consuming to debug. So this is an important feature, and it's worth understanding the background.

Setting the scene, with MyStringable

First, let's consider a simple implementation of `IStringable`.

C++/WinRT

```
struct MyStringable : implements<MyStringable, IStringable>
{
    winrt::hstring ToString() const { return L"MyStringable"; }
};
```

Now imagine that you need to call a function (from within your implementation) that expects an `IStringable` as an argument.

C++/WinRT

```
void Print(IStringable const& stringable)
{
    printf("%ls\n", stringable.ToString().c_str());
}
```

The trouble is that our `MyStringable` type is *not* an `IStringable`.

- Our `MyStringable` type is an implementation of the `IStringable` interface.
- The `IStringable` type is a projected type.

Important

It's important to understand the distinction between an *implementation type* and a *projected type*. For essential concepts and terms, be sure to read [Consume APIs](#)

The space between an implementation and the projection can be subtle to grasp. And in fact, to try to make the implementation feel a bit more like the projection, the implementation provides implicit conversions to each of the projected types that it implements. That doesn't mean we can simply do this.

C++/WinRT

```
struct MyStringable : implements<MyStringable, IStringable>
{
    winrt::hstring ToString() const;

    void Call()
    {
        Print(this);
    }
};
```

Instead, we need to get a reference so that conversion operators may be used as candidates for resolving the call.

C++/WinRT

```
void Call()
{
    Print(*this);
}
```

That works. An implicit conversion provides a (very efficient) conversion from the implementation type to the projected type, and that's very convenient for many scenarios. Without that facility, a lot of implementation types would prove very cumbersome to author. Provided that you only use the `winrt::make` function template (or `winrt::make_self`) to allocate the implementation, then all is well.

C++/WinRT

```
IStringable stringable{ winrt::make<MyStringable>() };
```

Potential pitfalls with C++/WinRT 1.0

Still, implicit conversions can land you in trouble. Consider this unhelpful helper function.

C++/WinRT

```
IStringable MakeStringable()
{
    return MyStringable(); // Incorrect.
}
```

Or even just this apparently harmless statement.

C++/WinRT

```
IStringable stringable{ MyStringable() }; // Also incorrect.
```

Unfortunately, code like that *did* compile with C++/WinRT 1.0, because of that implicit conversion. The (very serious) problem is that we're potentially returning a projected type that points to a reference-counted object whose backing memory is on the ephemeral stack.

Here's something else that compiled with C++/WinRT 1.0.

C++/WinRT

```
MyStringable* stringable{ new MyStringable() }; // Very inadvisable.
```

Raw pointers are dangerous and labor-intensive source of bugs. Don't use them if you don't need to. C++/WinRT goes out of its way to make everything efficient without ever forcing you into using raw pointers. Here's something else that compiled with C++/WinRT 1.0.

C++/WinRT

```
auto stringable{ std::make_shared<MyStringable>() }; // Also very
inadvisable.
```

This is a mistake on several levels. We have two different reference counts for the same object. The Windows Runtime (and classic COM before it) is based on an intrinsic reference count that's not compatible with `std::shared_ptr`. `std::shared_ptr` has, of course, many valid applications; but it's entirely unnecessary when you're sharing Windows Runtime (and classic COM) objects. Finally, this also compiled with C++/WinRT 1.0.

C++/WinRT

```
auto stringable{ std::make_unique<MyStringable>() }; // Highly dubious.
```

This is again rather questionable. The unique ownership is in opposition to the shared lifetime of the **MyStringable**'s intrinsic reference count.

The solution with C++/WinRT 2.0

With C++/WinRT 2.0, all of these attempts to directly allocate implementation types leads to a compiler error. That's the best kind of error, and infinitely better than a mysterious runtime bug.

Whenever you need to make an implementation, you can simply use `winrt::make` or `winrt::make_self`, as shown above. And now, if you forget to do so, then you'll be greeted with a compiler error alluding to this with a reference to an abstract function named `use_make_function_to_create_this_object`. It's not exactly a `static_assert`; but it's close. Still, this is the most reliable way of detecting all of the mistakes described.

It does mean that we need to place a few minor constraints on the implementation. Given that we're relying on the absence of an override to detect direct allocation, the `winrt::make` function template must somehow satisfy the abstract virtual function with an override. It does so by deriving from the implementation with a `final` class that provides the override. There are a few things to observe about this process.

First, the virtual function is only present in debug builds. Which means that detection isn't going to affect the size of the vtable in your optimized builds.

Second, since the derived class that `winrt::make` uses is `final`, it means that any devirtualization that the optimizer can possibly deduce will happen even if you previously chose not to mark your implementation class as `final`. So that's an improvement. The converse is that your implementation *can't* be `final`. Again, that's of no consequence because the instantiated type will always be `final`.

Third, nothing prevents you from marking any virtual functions in your implementation as `final`. Of course, C++/WinRT is very different from classic COM and implementations such as WRL, where everything about your implementation tends to be virtual. In C++/WinRT, the virtual dispatch is limited to the application binary interface (ABI) (which is always `final`), and your implementation methods rely on compile-time or static polymorphism. That avoids unnecessary runtime polymorphism, and also means that there's precious little reason for virtual functions in your C++/WinRT implementation. Which is a very good thing, and leads to far more predictable inlining.

Fourth, since `winrt::make` injects a derived class, your implementation can't have a private destructor. Private destructors were popular with classic COM implementations

because, again, everything was virtual, and it was common to deal directly with raw pointers and thus was easy to accidentally call `delete` instead of [Release](#). C++/WinRT goes out of its way to make it hard for you to deal directly with raw pointers. And you'd have to *really* go out of your way to get a raw pointer in C++/WinRT that you could potentially call `delete` on. Value semantics means that you're dealing with values and references; and rarely with pointers.

So, C++/WinRT challenges our preconceived notions of what it means to write classic COM code. And that's perfectly reasonable because WinRT is not classic COM. Classic COM is the assembly language of the Windows Runtime. It shouldn't be the code you write every day. Instead, C++/WinRT gets you to write code that's more like modern C++, and far less like classic COM.

Important APIs

- [winrt::make function template](#)
- [winrt::make_self function template](#)

Related topics

- [Consume APIs with C++/WinRT](#)
- [Author APIs with C++/WinRT](#)

Extension points for your implementation types

Article • 10/20/2022

The [winrt::implements struct template](#) is the base from which your own C++/WinRT implementations (of runtime classes and activation factories) directly or indirectly derive.

This topic discusses the extension points of **winrt::implements** in C++/WinRT 2.0. You can choose to implement these extension points on your implementation types, in order to customize the default behavior of inspectable objects (*inspectable* in the sense of the [IInspectable](#) interface).

These extension points allow you to defer destruction of your implementation types, to safely query during destruction, and to hook the entry into and exit from your projected methods. This topic describes those features and explains more about when and how you would use them.

Deferred destruction

In the [Diagnosing direct allocations](#) topic, we mentioned that your implementation type can't have a private destructor.

The benefit of having a public destructor is that it enables deferred destruction, which is the ability to detect the final [IUnknown::Release](#) call on your object, and then to take ownership of that object to defer its destruction indefinitely.

Recall that classic COM objects are intrinsically reference counted; the reference count is managed via the [IUnknown::AddRef](#) and [IUnknown::Release](#) functions. In a traditional implementation of **Release**, a classic COM object's C++ destructor is invoked once the reference count reaches 0.

C++/WinRT

```
uint32_t WINRT_CALL Release() noexcept
{
    uint32_t const remaining{ subtract_reference() };

    if (remaining == 0)
    {
        delete this;
    }
}
```

```
    return remaining;
}
```

The `delete this;` calls the object's destructor before freeing the memory occupied by the object. This works well enough, provided you don't need to do anything interesting in your destructor.

C++/WinRT

```
using namespace winrt::Windows::Foundation;
...
struct Sample : implements<Sample, IStringable>
{
    winrt::hstring ToString() const;

    ~Sample() noexcept
    {
        // Too late to do anything interesting.
    }
};
```

What do we mean by *interesting*? For one thing, a destructor is inherently synchronous. You can't switch threads—perhaps to destroy some thread-specific resources in a different context. You can't reliably query the object for some other interface that you might need in order to free certain resources. The list goes on. For the cases where your destruction is non-trivial, you need a more flexible solution. Which is where C++/WinRT's **final_release** function comes in.

C++/WinRT

```
struct Sample : implements<Sample, IStringable>
{
    winrt::hstring ToString() const;

    static void final_release(std::unique_ptr<Sample> ptr) noexcept
    {
        // This is the first stop...
    }

    ~Sample() noexcept
    {
        // ...And this happens only when *unique_ptr* finally deletes the
        object.
    }
};
```

We've updated the C++/WinRT implementation of **Release** to call your **final_release** right when your object's reference count transitions to 0. In that state, the object can be

confident that there are no further outstanding references, and it now has exclusive ownership of itself. For that reason, it can transfer ownership of itself to the static **final_release** function.

In other words, the object has transformed itself from one that supports shared ownership into one that is exclusively owned. The **std::unique_ptr** has exclusive ownership of the object, and so it'll naturally destroy the object as part of its semantics—hence the need for a public destructor—when the **std::unique_ptr** goes out of scope (provided that it isn't moved elsewhere before that). And that's the key. You can use the object indefinitely, provided that the **std::unique_ptr** keeps the object alive. Here's an illustration of how you might move the object elsewhere.

C++/WinRT

```
struct Sample : implements<Sample, IStringable>
{
    winrt::hstring ToString() const;

    static void final_release(std::unique_ptr<Sample> ptr) noexcept
    {
        batch_cleanup.push_back(std::move(ptr));
    }
};
```

This code saves the object in a collection named *batch_cleanup* one of whose jobs will be to clean up all the objects at some future point in the app's run-time.

Normally, the object destructs when the **std::unique_ptr** destructs, but you can hasten its destruction by calling **std::unique_ptr::reset**; or you can postpone it by saving the **std::unique_ptr** somewhere.

Perhaps more practically and more powerfully, you can turn the **final_release** function into a coroutine, and handle its eventual destruction in one place while being able to suspend and switch threads as needed.

C++/WinRT

```
struct Sample : implements<Sample, IStringable>
{
    winrt::hstring ToString() const;

    static winrt::fire_and_forget final_release(std::unique_ptr<Sample> ptr)
    noexcept
    {
        co_await winrt::resume_background(); // Unwind the calling thread.

        // Safely perform complex teardown here.
    }
};
```

```
    }  
};
```

A suspension will point causes the calling thread—which originally initiated the call to the **IUnknown::Release** function—to return, and thus signal to the caller that the object it once held is no longer available through that interface pointer. UI frameworks often need to ensure that objects are destroyed on the specific UI thread that originally created the object. This feature makes fulfilling such a requirement trivial, because destruction is separated from releasing the object.

Note that the object passed to **final_release** is merely a C++ object; it is no longer a COM object. For example, existing COM weak references to the object no longer resolve.

Safe queries during destruction

Building on the notion of deferred destruction is the ability to safely query for interfaces during destruction.

Classic COM is based on two central concepts. The first is reference counting, and the second is querying for interfaces. In addition to **AddRef** and **Release**, the **IUnknown** interface provides **QueryInterface**. That method is heavily used by certain UI frameworks—such as XAML, to traverse the XAML hierarchy as it simulates its composable type system. Consider a simple example.

C++/WinRT

```
struct MainPage : PageT<MainPage>  
{  
    ~MainPage()  
    {  
        DataContext(nullptr);  
    }  
};
```

That may *appear* harmless. This XAML page wants to clear its data context in its destructor. But **DataContext** is a property of the **FrameworkElement** base class, and it lives on the distinct **IFrameworkElement** interface. As a result, C++/WinRT must inject a call to **QueryInterface** to look up the correct vtable before being able to call the **DataContext** property. But the reason we're even in the destructor is that the reference count has transitioned to 0. Calling **QueryInterface** here temporarily bumps that reference count; and when it again returns to 0, the object destructs again.

C++/WinRT 2.0 has been hardened to support this. Here's the C++/WinRT 2.0 implementation of `Release`, in a simplified form.

C++/WinRT

```
uint32_t Release() noexcept
{
    uint32_t const remaining{ subtract_reference() };

    if (remaining == 0)
    {
        m_references = 1; // Debouncing!
        T::final_release(...);
    }

    return remaining;
}
```

As you might have predicted, it first decrements the reference count, and then acts only if there are no outstanding references. However, before calling the static **final_release** function that we described earlier in this topic, it stabilizes the reference count by setting it to 1. We refer to this as *debouncing* (borrowing a term from electrical engineering). This is critical to prevent the final reference from being release. Once that happens, the reference count is unstable, and isn't able to reliably support a call to **QueryInterface**.

Calling **QueryInterface** is dangerous after the final reference has been released, because the reference count can then conceivably grow indefinitely. It's your responsibility to call only known code paths that won't prolong the life of the object. C++/WinRT meets you halfway by ensuring that those **QueryInterface** calls can be made reliably.

It does that by stabilizing the reference count. When the final reference has been released, the actual reference count is either 0, or some wildly unpredictable value. The latter case may occur if weak references are involved. Either way, this is unsustainable if a subsequent call to **QueryInterface** occurs; because that will necessarily cause the reference count to increment temporarily—hence the reference to debouncing. Setting it to 1 ensures that a final call to **Release** will never again occur on this object. That's precisely what we want, since the **std::unique_ptr** now owns the object, but bounded calls to **QueryInterface/Release** pairs will be safe.

Consider a more interesting example.

C++/WinRT

```
struct MainPage : PageT<MainPage>
{
```

```

    ~MainPage()
    {
        DataContext(nullptr);
    }

    static winrt::fire_and_forget final_release(std::unique_ptr<MainPage>
ptr)
    {
        co_await 5s;
        co_await winrt::resume_foreground(ptr->Dispatcher());
        ptr = nullptr;
    }
};

```

First, the **final_release** function is called, notifying the implementation that it's time to clean up. Here, **final_release** happens to be a coroutine. To simulate a first suspension point, it begins by waiting on the thread pool for a few seconds. It then resumes on the page's dispatcher thread. That last step involves a query, since **Dispatcher** is a property of the **DependencyObject** base class. Finally, the page is actually deleted by virtue of assigning `nullptr` to the `std::unique_ptr`. That in turn calls the page's destructor.

Inside the destructor, we clear the data context; which, as we know, requires a query for the **FrameworkElement** base class.

All of this possible because of the reference count debouncing (or reference count stabilization) provided by C++/WinRT 2.0.

Method entry and exit hooks

A less commonly-used extension point is the **abi_guard** struct, and the **abi_enter** and **abi_exit** functions.

If your implementation type defines a function **abi_enter**, then that function is called at the entry to every one of your projected interface methods (not counting the methods of **IInspectable**).

Similarly, if you define **abi_exit**, then that will be called at the exit from every such method; but it will not be called if your **abi_enter** throws an exception. It *will* still be called if an exception is thrown by your projected interface method itself.

As an example, you might use **abi_enter** to throw a hypothetical **invalid_state_error** exception if a client tries to use an object after the object has been put into an unusable state—say after a **ShutDown** or **Disconnect** method call. The C++/WinRT iterator classes use this feature to throw an invalid state exception in the **abi_enter** function if the underlying collection has changed.

Over and above the simple **abi_enter** and **abi_exit** functions, you can define a nested type named **abi_guard**. In that case, an instance of **abi_guard** is created on entry to each (non-**Inspectable**) of your projected interface methods, with a reference to the object as its constructor parameter. The **abi_guard** is then destructured on exit from the method. You can put whatever extra state you like into your **abi_guard** type.

If you don't define your own **abi_guard**, then there's a default one that calls **abi_enter** at construction, and **abi_exit** at destruction.

These guards are used only when a method is invoked *via the projected interface*. If you invoke methods directly on the implementation object, then those calls go straight to the implementation, without any guards.

Here's a code example.

C++/WinRT

```
struct Sample : SampleT<Sample, IClosable>
{
    void abi_enter();
    void abi_exit();

    void Close();
};

void example1()
{
    auto sampleObj1{ winrt::make<Sample>() };
    sampleObj1.Close(); // Calls abi_enter and abi_exit.
}

void example2()
{
    auto sampleObj2{ winrt::make_self<Sample>() };
    sampleObj2->Close(); // Doesn't call abi_enter nor abi_exit.
}

// A guard is used only for the duration of the method call.
// If the method is a coroutine, then the guard applies only until
// the IAsyncXxx is returned; not until the coroutine completes.

IAsyncAction CloseAsync()
{
    // Guard is active here.
    DoWork();

    // Guard becomes inactive once DoOtherWorkAsync
    // returns an IAsyncAction.
    co_await DoOtherWorkAsync();
}
```

```
} // Guard is not active here.
```


A basic C++/WinRT Windows UI Library 2 example (UWP)

Article • 10/20/2022

This topic walks you through the process of adding basic support for the [Windows UI Library \(WinUI\)](#) to your C++/WinRT UWP project. Specifically, this topic deals with WinUI 2, which is for UWP apps. For desktop apps, there's [WinUI 3](#). Incidentally, WinUI is itself written in C++/WinRT.

❗ Important

For desktop apps, the [Windows App SDK](#) provides [Windows UI Library \(WinUI\) 3](#). WinUI 3 is not intended to work with the walkthrough in this topic, which is for UWP. Also see [Migrate from UWP to the Windows App SDK](#).

ⓘ Note

The Windows UI Library (WinUI) toolkit is available as NuGet packages that you can add to any existing or new project using Visual Studio, as we'll see in this topic. For more background, setup, and support info, see [Getting started with the Windows UI Library](#).

Create a Blank App (HelloWinUICppWinRT)

In Visual Studio, create a new UWP project using the **Blank App (C++/WinRT)** project template. Make sure that you're using the (C++/WinRT) template, and not the (Universal Windows) one.

Set the name of the new project to *HelloWinUICppWinRT*, and (so that your folder structure will match the walkthrough) uncheck **Place solution and project in the same directory**.

Install the Microsoft.UI.Xaml NuGet package

Click **Project > Manage NuGet Packages... > Browse**, type or paste **Microsoft.UI.Xaml** in the search box, select the item in search results, and then click **Install** to install the

package into your project (you'll also see a license agreement prompt). Be careful to install only the **Microsoft.UI.Xaml** package, and not **Microsoft.UI.Xaml.Core.Direct**.

Declare WinUI application resources

Open `App.xaml` and paste the following markup between the existing opening and closing **Application** tags.

XAML

```
<Application.Resources>
    <XamlControlsResources xmlns="using:Microsoft.UI.Xaml.Controls"/>
</Application.Resources>
```

Add a WinUI control to MainPage

Next, open `MainPage.xaml`. In the existing opening **Page** tag there are some xml namespace declarations. Add the xml namespace declaration `xmlns:muxc="using:Microsoft.UI.Xaml.Controls"`. Then, paste the following markup between the existing opening and closing **Page** tags, overwriting the existing **StackPanel** element.

XAML

```
<muxc:NavigationView PaneTitle="Welcome">
    <TextBlock Text="Hello, World!" VerticalAlignment="Center"
HorizontalAlignment="Center" Style="{StaticResource TitleTextBlockStyle}"/>
</muxc:NavigationView>
```

Edit pch.h, as necessary

When you add a NuGet package to a C++/WinRT project (such as the **Microsoft.UI.Xaml** package, which you added earlier), and build the project, the tooling generates a set of projection header files in your project's `\Generated Files\winrt` folder. If you've followed along with the walkthrough, you'll now have a `\HelloWinUICppWinRT\HelloWinUICppWinRT\Generated Files\winrt` folder. To bring those headers files into your project, so that references to those new types resolve, you can go into your precompiled header file (typically `pch.h`) and include them.

You need to include only the headers that correspond to the types that you use. But here's an example that includes all of the generated header files for the

Microsoft.UI.Xaml package.

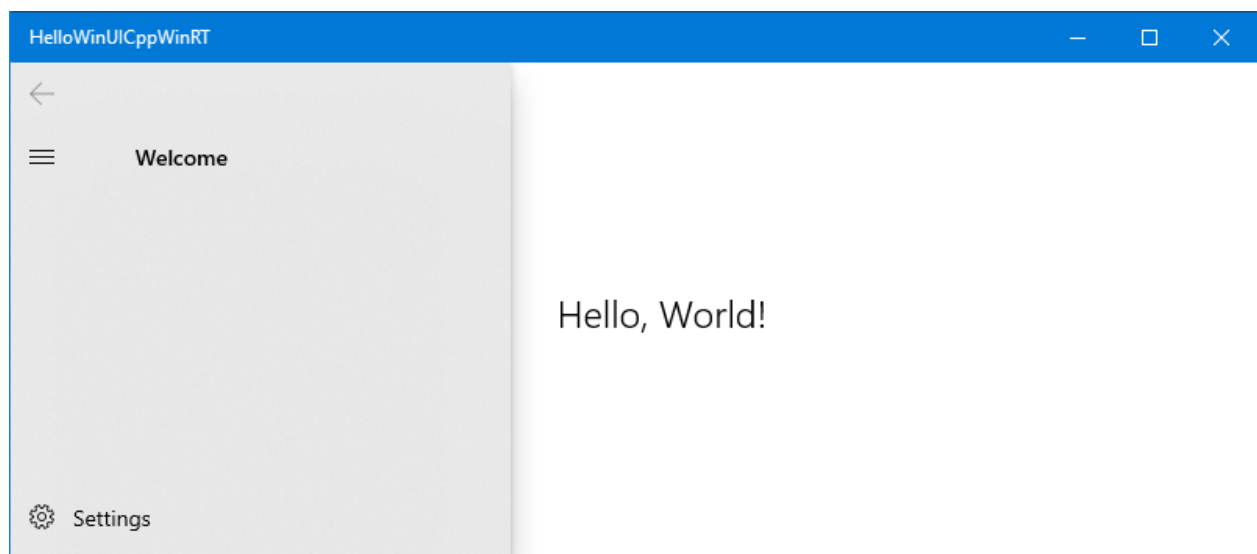
C++/WinRT

```
// pch.h
...
#include "winrt/Microsoft.UI.Xaml.Automation.Peers.h"
#include "winrt/Microsoft.UI.Xaml.Controls.h"
#include "winrt/Microsoft.UI.Xaml.Controls.Primitives.h"
#include "winrt/Microsoft.UI.Xaml.Media.h"
#include "winrt/Microsoft.UI.Xaml.XamlTypeInfo.h"
...
```

Edit MainPage.cpp

In `MainPage.cpp`, delete the code inside your implementation of `MainPage::ClickHandler`, since *myButton* is no longer in the XAML markup.

You can now build and run the project.



Related topics

- [Getting started with the Windows UI Library](#)
- [Windows App SDK](#)
- [Windows UI Library \(WinUI\) 3](#)
- [Migrate from UWP to the Windows App SDK](#)

Windows Runtime components with C++/WinRT

Article • 10/20/2022

This topic shows how to use [C++/WinRT](#) to create and consume a Windows Runtime component—a component that's callable from a Universal Windows app built using any Windows Runtime language.

There are several reasons for building a Windows Runtime component in C++/WinRT.

- To enjoy the performance advantage of C++ in complex or computationally intensive operations.
- To reuse standard C++ code that's already written and tested.
- To expose Win32 functionality to a Universal Windows Platform (UWP) app written in, for example, C#.

In general, when you author your C++/WinRT component, you can use types from the standard C++ library, and built-in types, except at the application binary interface (ABI) boundary where you're passing data to and from code in another `.winmd` package. At the ABI, use Windows Runtime types. In addition, in your C++/WinRT code, use types such as delegate and event to implement events that can be raised from your component and handled in another language. See [C++/WinRT](#) for more info about C++/WinRT.

The remainder of this topic walks you through how to author a Windows Runtime component in C++/WinRT, and then how to consume it from an application.

The Windows Runtime component that you'll build in this topic contains a runtime class representing a thermometer. The topic also demonstrates a Core App that consumes the thermometer runtime class, and calls a function to adjust the temperature.

ⓘ Note

For info about installing and using the [C++/WinRT Visual Studio Extension \(VSIX\)](#) and the NuGet package (which together provide project template and build support), see [Visual Studio support for C++/WinRT](#).

ⓘ Important

For essential concepts and terms that support your understanding of how to consume and author runtime classes with C++/WinRT, see [Consume APIs with C++/WinRT](#) and [Author APIs with C++/WinRT](#).

Naming best practice for Windows Runtime component dlls

Important

This section describes the naming convention that we recommend you use for the `.dll` file (DLL) that you build your Windows Runtime component into. It's all about the activation sequence that C++/WinRT follows when you consume a runtime class from a Windows Runtime component.

When activating a class factory, C++/WinRT first tries a call to [RoGetActivationFactory](#). If that fails, then C++/WinRT tries to find a DLL to load directly. Windows Runtime activation is always based on a fully-qualified class name. The logic is to remove the class name (from that fully-qualified class name), and then look for a DLL named for the full namespace that remains. If that's not found, remove the most-specific segment name, and repeat.

So for example if the class being activated has a fully-qualified name of **Contoso.Instruments.ThermometerWRC.Thermometer**, and [RoGetActivationFactory](#) fails, then we'll first look for a `Contoso.Instruments.ThermometerWRC.dll`. If that's not found, then we'll look for `Contoso.Instruments.dll`, and then for `Contoso.dll`.

When a DLL is found (in that sequence), we'll use that DLL's [DllGetActivationFactory](#) entry point to try to obtain the factory directly (rather than indirectly via the [RoGetActivationFactory](#) function that we first attempted). Even so, the end result is indistinguishable to the caller and to the DLL.

This process is completely automatic—no registration or tooling is needed. If you're authoring a Windows Runtime component, then you just need to use a naming convention for your DLLs that works with the process just described. And if you're consuming a Windows Runtime component and it's not named correctly, then you have the option to rename it as described.

Create a Windows Runtime component (ThermometerWRC)

Begin by creating a new project in Microsoft Visual Studio. Create a **Windows Runtime Component (C++/WinRT)** project, and name it *ThermometerWRC* (for "thermometer Windows Runtime component"). Make sure that **Place solution and project in the same directory** is unchecked. Target the latest generally-available (that is, not preview) version of the Windows SDK. Naming the project *ThermometerWRC* will give you the easiest experience with the rest of the steps in this topic.

Don't build the project yet.

The newly-created project contains a file named `Class.idl`. In Solution Explorer, rename that file `Thermometer.idl` (renaming the `.idl` file automatically renames the dependent `.h` and `.cpp` files, too). Replace the contents of `Thermometer.idl` with the listing below.

idl

```
// Thermometer.idl
namespace ThermometerWRC
{
    runtimeclass Thermometer
    {
        Thermometer();
        void AdjustTemperature(Single deltaFahrenheit);
    };
}
```

Save the file. The project won't build to completion at the moment, but building now is a useful thing to do because it generates the source code files in which you'll implement the **Thermometer** runtime class. So go ahead and build now (the build errors you can expect to see at this stage have to do with `Class.h` and `Class.g.h` not being found).

During the build process, the `midl.exe` tool is run to create your component's Windows Runtime metadata file (which is

`\ThermometerWRC\Debug\ThermometerWRC\ThermometerWRC.winmd`). Then, the `cppwinrt.exe` tool is run (with the `-component` option) to generate source code files to support you in authoring your component. These files include stubs to get you started implementing the **Thermometer** runtime class that you declared in your IDL. Those stubs are `\ThermometerWRC\ThermometerWRC\Generated Files\sources\Thermometer.h` and `Thermometer.cpp`.

Right-click the project node and click **Open Folder in File Explorer**. This opens the project folder in File Explorer. There, copy the stub files `Thermometer.h` and `Thermometer.cpp` from the folder `\ThermometerWRC\ThermometerWRC\Generated Files\sources\` and into the folder that contains your project files, which is `\ThermometerWRC\ThermometerWRC\`, and replace the files in the destination. Now, let's open `Thermometer.h` and `Thermometer.cpp` and implement our runtime class. In `Thermometer.h`, add a new private member to the implementation (*not* the factory implementation) of **Thermometer**.

C++/WinRT

```
// Thermometer.h
...
namespace winrt::ThermometerWRC::implementation
{
    struct Thermometer : ThermometerT<Thermometer>
    {
        ...

        private:
            float m_temperatureFahrenheit { 0.f };
    };
}
```

In `Thermometer.cpp`, implement the **AdjustTemperature** method as shown in the listing below.

C++/WinRT

```
// Thermometer.cpp
...
namespace winrt::ThermometerWRC::implementation
{
    void Thermometer::AdjustTemperature(float deltaFahrenheit)
    {
        m_temperatureFahrenheit += deltaFahrenheit;
    }
}
```

You'll see a `static_assert` at the top of `Thermometer.h` and `Thermometer.cpp`, which you'll need to remove. Now the project will build.

If any warnings prevent you from building, then either resolve them or set the project property **C/C++ > General > Treat Warnings As Errors** to **No (/WX-)**, and build the project again.

Create a Core App (ThermometerCoreApp) to test the Windows Runtime component

Now create a new project (either in your *ThermometerWRC* solution, or in a new one). Create a **Core App (C++/WinRT)** project, and name it *ThermometerCoreApp*. Set *ThermometerCoreApp* as the startup project if the two projects are in the same solution.

❗ Note

As mentioned earlier, the Windows Runtime metadata file for your Windows Runtime component (whose project you named *ThermometerWRC*) is created in the folder `\ThermometerWRC\Debug\ThermometerWRC\`. The first segment of that path is the name of the folder that contains your solution file; the next segment is the subdirectory of that named `Debug`; and the last segment is the subdirectory of that named for your Windows Runtime component. If you didn't name your project *ThermometerWRC*, then your metadata file will be in the folder `\<YourProjectName>\Debug\<YourProjectName>\`.

Now, in your Core App project (*ThermometerCoreApp*), add a reference, and browse to `\ThermometerWRC\Debug\ThermometerWRC\ThermometerWRC.winmd` (or add a project-to-project reference, if the two projects are in the same solution). Click **Add**, and then **OK**. Now build *ThermometerCoreApp*. In the unlikely event that you see an error that the payload file `readme.txt` doesn't exist, exclude that file from the Windows Runtime component project, rebuild it, then rebuild *ThermometerCoreApp*.

During the build process, the `cppwinrt.exe` tool is run to process the referenced `.winmd` file into source code files containing projected types to support you in consuming your component. The header for the projected types for your component's runtime classes—named `ThermometerWRC.h`—is generated into the folder `\ThermometerCoreApp\ThermometerCoreApp\Generated Files\winrt\`.

Include that header in `App.cpp`.

C++/WinRT

```
// App.cpp
...
#include <winrt/ThermometerWRC.h>
...
```


Also in `App.cpp`, add the following code to instantiate a **Thermometer** object (using the projected type's default constructor), and call a method on the thermometer object.

C++/WinRT

```
struct App : implements<App, IFrameworkViewSource, IFrameworkView>
{
    ThermometerWRC::Thermometer m_thermometer;
    ...

    void OnPointerPressed(IInspectable const &, PointerEventArgs const &
args)
    {
        m_thermometer.AdjustTemperature(1.f);
        ...
    }
    ...
};
```

Each time you click the window, you increment the thermometer object's temperature. You can set breakpoints if you want to step through the code to confirm that the application really is calling into the Windows Runtime component.

Next steps

To add even more functionality, or new Windows Runtime types, to your C++/WinRT Windows Runtime component, you can follow the same patterns shown above. First, use IDL to define the functionality you want to expose. Then build the project in Visual Studio to generate a stub implementation. And then complete the implementation as appropriate. Any methods, properties, and events that you define in IDL are visible to the application that consumes your Windows Runtime Component. For more info about IDL, see [Introduction to Microsoft Interface Definition Language 3.0](#).

For an example of how to add an event to your Windows Runtime Component, see [Author events in C++/WinRT](#).

Troubleshooting

Symptom	Remedy
---------	--------

Symptom	Remedy
In a C++/WinRT app, when consuming a C# Windows Runtime component that uses XAML, the compiler produces an error of the form "'MyNamespace_XamlTypeInfo': is not a member of 'winrt::MyNamespace'"—where <i>MyNamespace</i> is the name of the Windows Runtime component's namespace.	In <code>pch.h</code> in the consuming C++/WinRT app, add <code>#include <winrt/MyNamespace.MyNamespace_XamlTypeInfo.h></code> —replacing <i>MyNamespace</i> as appropriate.

Authoring a C# Windows Runtime component for use from a C++/WinRT app

Article • 10/27/2022

This topic walks you through the process of adding a simple C# component to your C++/WinRT project.

Visual Studio makes it easy to author and deploy your own custom Windows Runtime types inside a Windows Runtime component (WRC) project written with C# or Visual Basic, and then to reference that WRC from a C++ application project, and to consume those custom types from that application.

Internally, your Windows Runtime types can use any .NET functionality that's allowed in a UWP application.

ⓘ Note

For more info, see [Windows Runtime components with C# and Visual Basic and .NET for UWP apps overview](#).

Externally, the members of your type can expose only Windows Runtime types for their parameters and return values. When you build your solution, Visual Studio builds your .NET WRC project, and then executes a build step that creates a Windows metadata (.winmd) file. This is your Windows Runtime component (WRC), which Visual Studio includes in your app.

ⓘ Note

.NET automatically maps some commonly used .NET types, such as primitive data types and collection types, to their Windows Runtime equivalents. These .NET types can be used in the public interface of a Windows Runtime component, and will appear to users of the component as the corresponding Windows Runtime types. See [Windows Runtime components with C# and Visual Basic](#).

Prerequisites

- Windows 10

- [Microsoft Visual Studio](#) ↗

Create a Blank App

In Visual Studio, create a new project using the **Blank App (C++/WinRT)** project template. Make sure that you're using the (C++/WinRT) template, and not the (Universal Windows) one.

Set the name of the new project to *CppToCSharpWinRT* so that your folder structure will match the walkthrough.

Add a C# Windows Runtime Component to the Solution

In Visual Studio, create the component project: In Solution Explorer, open the shortcut menu for the *CppToCSharpWinRT* solution and choose **Add**, and then choose **New Project** to add a new C# project to the solution. In the **Installed Templates** section of the **Add New Project** dialog box, choose **Visual C#**, and then choose **Windows**, and then **Universal**. Choose the **Windows Runtime Component (Universal Windows)** template and enter **SampleComponent** for the project name.

ⓘ Note

On the **New Universal Windows Platform Project** dialog box, choose **Windows 10 Creators Update (10.0; Build 15063)** as the Minimum Version. Please see the **Application Minimum Version** section below for more information.

Add the C# GetMyString method

In the *SampleComponent* project, change the name of the class from *Class1* to *Example*. Then add two simple members to the class, a private `int` field and an instance method named *GetMyString*:

C#

```
public sealed class Example
{
    int MyNumber;

    public string GetMyString()
    {
```

```
        return $"This is call #: {++MyNumber}";  
    }  
}
```

ⓘ Note

By default, the class is marked **public sealed**. All the Windows Runtime classes you expose from your component must be **sealed**.

ⓘ Note

Optional: To enable IntelliSense for the newly added members, in Solution Explorer, open the shortcut menu for the *SampleComponent* project, and then choose **Build**.

Reference the C# SampleComponent from the CppToCSharpWinRT project

In Solution Explorer, in the C++/WinRT project, open the shortcut menu for **References**, and then choose **Add Reference** to open the **Add Reference** dialog. Choose **Projects**, and then choose **Solution**. Select the check box for the *SampleComponent* project and choose **OK** to add a reference.

ⓘ Note

Optional: To enable IntelliSense for the C++/WinRT project, in Solution Explorer, open the shortcut menu for the *CppToCSharpWinRT* project, and then choose **Build**.

Edit MainPage.h

Open `MainPage.h` in the *CppToCSharpWinRT* project and then add two items. First add `#include "winrt/SampleComponent.h"` at the end of the `#include` statements, then a `winrt::SampleComponent::Example` field to the `MainPage` struct.

C++/WinRT

```
// MainPage.h  
...  
#include "winrt/SampleComponent.h"
```

```
namespace winrt::CppToCSharpWinRT::implementation
{
    struct MainPage : MainPageT<MainPage>
    {
        ...
        winrt::SampleComponent::Example myExample;
        ...
    };
}
```

ⓘ Note

In Visual Studio, `MainPage.h` is listed under `MainPage.xaml`.

Edit MainPage.cpp

In `MainPage.cpp`, change the `Mainpage::ClickHandler` implementation to call the C# method `GetMyString`.

C++/WinRT

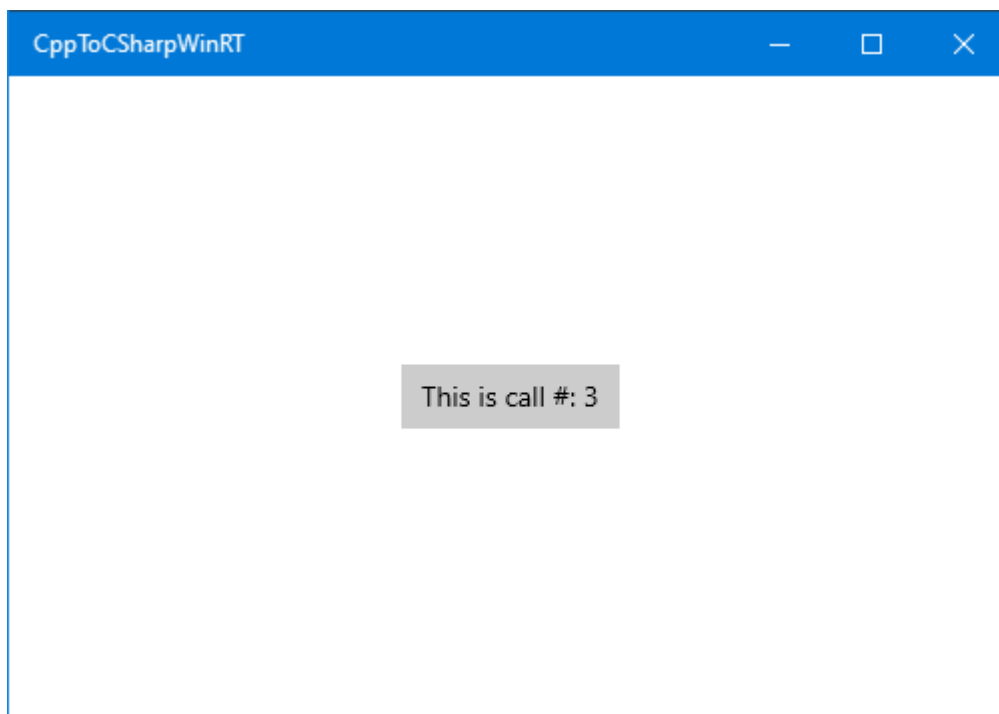
```
void MainPage::ClickHandler(IInspectable const&, RoutedEventArgs const&)
{
    //myButton().Content(box_value(L"Clicked"));

    hstring myString = myExample.GetMyString();

    myButton().Content(box_value(myString));
}
```

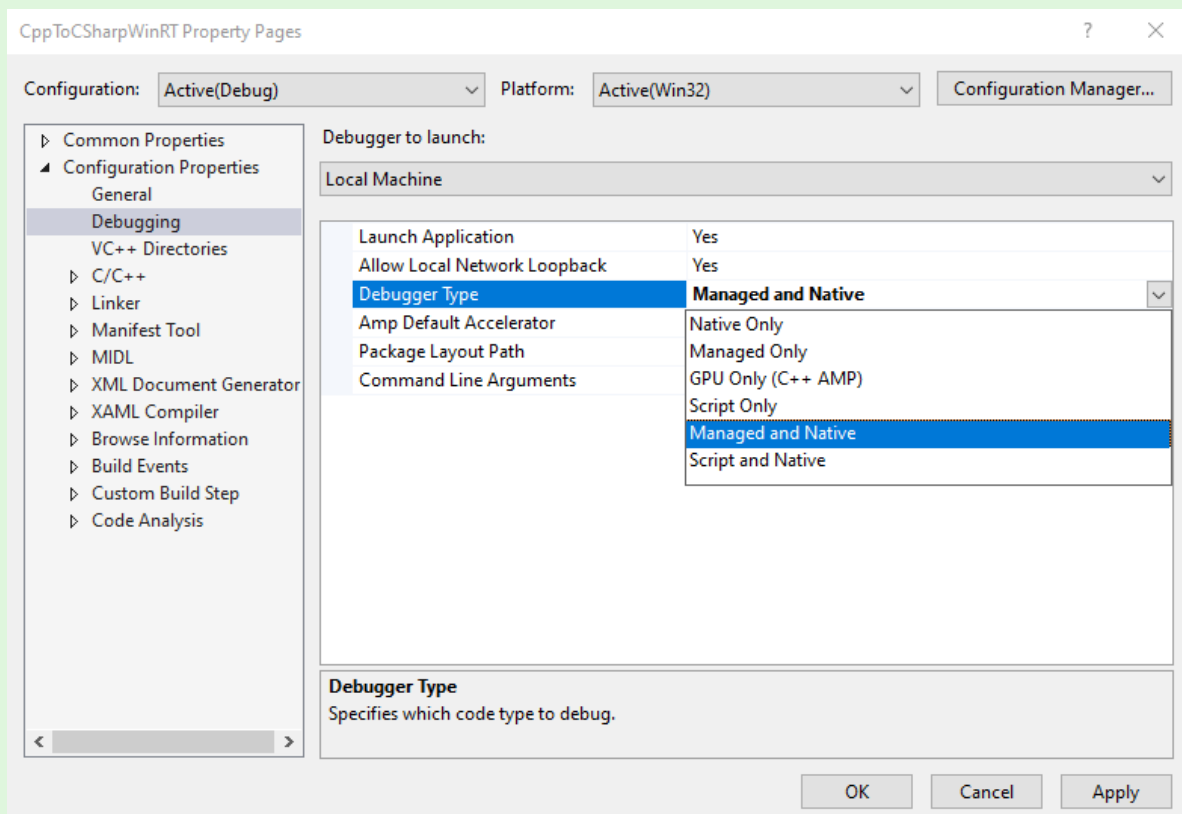
Run the project

You can now build and run the project. Each time you click the button, the number in the button will increment.



💡 Tip

In Visual Studio, create the component project: In Solution Explorer, open the shortcut menu for the *CppToCSharpWinRT* project and choose **Properties**, and then choose **Debugging** under **Configuration Properties**. Set the Debugger Type to *Managed and Native* if you want to debug both the C# (managed) and C++ (native) code.



Application Minimum Version

The **Application Minimum** of the C# project version will control the version of .NET used to compile the application. For example, choosing **Windows 10 Fall Creators Update (10.0; Build 16299)** or later will enable .NET Standard 2.0 and Windows Arm64 processor support.

Tip

We recommend using **Application Minimum** versions lower than 16299 to avoid extra build configuration if .NET Standard 2.0 or Arm64 support is not necessary.

Configure for Windows 10 Fall Creators Update (10.0; Build 16299)

Follow these steps to enable .NET Standard 2.0 or Windows Arm64 support in the C# projects referenced from your C++/WinRT project.

In Visual Studio, go to the Solution Explorer and open the shortcut menu for the *CppToCSharpWinRT* project. Choose **Properties** and set the Universal Windows App Min version to **Windows 10 Fall Creators Update (10.0; Build 16299)** (or higher). Do the same for the *SampleComponent* project.

In Visual Studio, open the shortcut menu for the *CppToCSharpWinRT* project and choose **Unload Project** to open `CppToCSharpWinRT.vcxproj` in the text editor.

Copy and paste the following XML to the first `PropertyGroup` in

`CPPWinRTCSharpV2.vcxproj`.

XML

```
<!-- Start Custom .NET Native properties -->
<DotNetNativeVersion>2.2.12-rel-31116-00</DotNetNativeVersion>
<DotNetNativeSharedLibrary>2.2.8-rel-31116-00</DotNetNativeSharedLibrary>
<UWPCoreRuntimeSdkVersion>2.2.14</UWPCoreRuntimeSdkVersion>
<!--<NuggetPath>$(USERPROFILE)\.nuget\packages</NuggetPath>-->
<NuggetPath>$(ProgramFiles)\Microsoft SDKs\UWPNuGetPackages</NuggetPath>
<!-- End Custom .NET Native properties -->
```

The values for `DotNetNativeVersion`, `DotNetNativeSharedLibrary`, and `UWPCoreRuntimeSdkVersion` may vary depending on the version of Visual Studio. To set them to the correct values, open the `%ProgramFiles(x86)%\Microsoft`

SDKs\UWPNuGetPackages and look at the sub-directory for each value in the table below. The %ProgramFiles(x86)%\Microsoft SDKs\UWPNuGetPackages\Microsoft.Net.Native.Compiler directory will have a sub-directory that contains an installed version of .NET native that starts with 2.2. In the example below, it is 2.2.12-rel-31116-00.

 Expand table

MSBuild Variable	Directory	Example
DotNetNativeVersion	%ProgramFiles(x86)%\Microsoft SDKs\UWPNuGetPackages\Microsoft.Net.Native.Compiler	2.2.12- rel- 31116-00
DotNetNativeSharedLibrary	%ProgramFiles(x86)%\Microsoft SDKs\UWPNuGetPackages\runtime.win10- x64.microsoft.net.native.sharedlibrary	2.2.8- rel- 31116-00
UWPCoreRuntimeSdkVersion	%ProgramFiles(x86)%\Microsoft SDKs\UWPNuGetPackages\Microsoft.Net.UWPCoreRuntimeSdk	2.2.14

Note

There are multiple supported architectures for Microsoft.Net.Native.SharedLibrary. Replace x64 with the appropriate architecture. For example, the arm64 architecture would be in the %ProgramFiles(x86)%\Microsoft SDKs\UWPNuGetPackages\runtime.win10-arm64.microsoft.net.native.sharedlibrary directory.

Next, immediately after the first PropertyGroup, add the following (unaltered).

XML

```
<!-- Start Custom .NET Native targets -->
<!-- Import all of the .NET Native / CoreCLR props at the beginning of the
project -->
<Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'"
Project="$(NuGetPath)\Microsoft.Net.UWPCoreRuntimeSdk\$(UWPCoreRuntimeSdkVer
sion)\build\Microsoft.Net.UWPCoreRuntimeSdk.props" />
<Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NuGetPath)\runtime.win10-
x86.Microsoft.Net.UWPCoreRuntimeSdk\$(UWPCoreRuntimeSdkVersion)\build\runtim
e.win10-x86.Microsoft.Net.UWPCoreRuntimeSdk.props" />
<Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
```

```

'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
x64.Microsoft.Net.UWPCoreRuntimeSdk\$(UWPCoreRuntimeSdkVersion)\build\runtim
e.win10-x64.Microsoft.Net.UWPCoreRuntimeSdk.props" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
arm.Microsoft.Net.UWPCoreRuntimeSdk\$(UWPCoreRuntimeSdkVersion)\build\runtim
e.win10-arm.Microsoft.Net.UWPCoreRuntimeSdk.props" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'"
Project="$(NugetPath)\Microsoft.Net.Native.Compiler\$(DotNetNativeVersion)\b
uild\Microsoft.Net.Native.Compiler.props" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
x86.Microsoft.Net.Native.Compiler\$(DotNetNativeVersion)\build\runtime.win10
-x86.Microsoft.Net.Native.Compiler.props" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
x64.Microsoft.Net.Native.Compiler\$(DotNetNativeVersion)\build\runtime.win10
-x64.Microsoft.Net.Native.Compiler.props" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
arm.Microsoft.Net.Native.Compiler\$(DotNetNativeVersion)\build\runtime.win10
-arm.Microsoft.Net.Native.Compiler.props" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
arm64.Microsoft.Net.Native.Compiler\$(DotNetNativeVersion)\build\runtime.win
10-arm64.Microsoft.Net.Native.Compiler.props" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
x86.Microsoft.Net.Native.SharedLibrary\$(DotNetNativeSharedLibrary)\build\ru
ntime.win10-x86.Microsoft.Net.Native.SharedLibrary.props" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
x64.Microsoft.Net.Native.SharedLibrary\$(DotNetNativeSharedLibrary)\build\ru
ntime.win10-x64.Microsoft.Net.Native.SharedLibrary.props" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
arm.Microsoft.Net.Native.SharedLibrary\$(DotNetNativeSharedLibrary)\build\ru
ntime.win10-arm.Microsoft.Net.Native.SharedLibrary.props" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
arm64.Microsoft.Net.Native.SharedLibrary\$(DotNetNativeSharedLibrary)\build\
runtime.win10-arm64.Microsoft.Net.Native.SharedLibrary.props" />
  <!-- End Custom .NET Native targets -->

```

At the end of the project file, just before the closing `Project` tag, add the following (unaltered).

XML

```

  <!-- Import all of the .NET Native / CoreCLR targets at the end of the
project -->
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=

```

```

'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
x86.Microsoft.Net.UWPCoreRuntimeSdk\$(UWPCoreRuntimeSdkVersion)\build\runtim
e.win10-x86.Microsoft.Net.UWPCoreRuntimeSdk.targets" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
x64.Microsoft.Net.UWPCoreRuntimeSdk\$(UWPCoreRuntimeSdkVersion)\build\runtim
e.win10-x64.Microsoft.Net.UWPCoreRuntimeSdk.targets" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
arm.Microsoft.Net.UWPCoreRuntimeSdk\$(UWPCoreRuntimeSdkVersion)\build\runtim
e.win10-arm.Microsoft.Net.UWPCoreRuntimeSdk.targets" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'"
Project="$(NugetPath)\Microsoft.Net.Native.Compiler\$(DotNetNativeVersion)\b
uild\Microsoft.Net.Native.Compiler.targets" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
x86.Microsoft.Net.Native.Compiler\$(DotNetNativeVersion)\build\runtime.win10
-x86.Microsoft.Net.Native.Compiler.targets" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
x64.Microsoft.Net.Native.Compiler\$(DotNetNativeVersion)\build\runtime.win10
-x64.Microsoft.Net.Native.Compiler.targets" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
arm.Microsoft.Net.Native.Compiler\$(DotNetNativeVersion)\build\runtime.win10
-arm.Microsoft.Net.Native.Compiler.targets" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
arm64.Microsoft.Net.Native.Compiler\$(DotNetNativeVersion)\build\runtime.win
10-arm64.Microsoft.Net.Native.Compiler.targets" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
x86.Microsoft.Net.Native.SharedLibrary\$(DotNetNativeSharedLibrary)\build\ru
ntime.win10-x86.Microsoft.Net.Native.SharedLibrary.targets" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
x64.Microsoft.Net.Native.SharedLibrary\$(DotNetNativeSharedLibrary)\build\ru
ntime.win10-x64.Microsoft.Net.Native.SharedLibrary.targets" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
arm.Microsoft.Net.Native.SharedLibrary\$(DotNetNativeSharedLibrary)\build\ru
ntime.win10-arm.Microsoft.Net.Native.SharedLibrary.targets" />
  <Import Condition="'$(WindowsTargetPlatformMinVersion)' >=
'10.0.16299.0'" Project="$(NugetPath)\runtime.win10-
arm64.Microsoft.Net.Native.SharedLibrary\$(DotNetNativeSharedLibrary)\build\
runtime.win10-arm64.Microsoft.Net.Native.SharedLibrary.targets" />
  <!-- End Custom .NET Native targets -->

```

Reload the project file in Visual Studio. To do this, in the Visual Studio Solution Explorer, open the shortcut menu for the *CppToCSharpWinRT* project and choose **Reload Project**.

Building for .NET Native

It is recommended to build and test your application with the C# component built against .NET native. In Visual Studio, open the shortcut menu for the *CppToCSharpWinRT* project and choose **Unload Project** to open `CppToCSharpWinRT.vcxproj` in the text editor.

Next, set the `UseDotNetNativeToolchain` property to `true` in the Release and Arm64 configurations in the C++ project file.

In the Visual Studio Solution Explorer, open the shortcut menu for the *CppToCSharpWinRT* project and choose **Reload Project**.

XML

```
<PropertyGroup Condition="'$(Configuration)'=='Release'"
Label="Configuration">
...
<UseDotNetNativeToolchain>true</UseDotNetNativeToolchain>
</PropertyGroup>
<PropertyGroup Condition="'$(Platform)'=='Arm64'" Label="Configuration">
<UseDotNetNativeToolchain
Condition="'$(UseDotNetNativeToolchain)'==''">true</UseDotNetNativeToolchain
>
</PropertyGroup>
```

Referencing other C# nuget packages

If the C# component is referencing other [nuget packages](#), the application's project file may need list file dependencies from the nuget package as deployment content. For example, if the C# component references the Newtonsoft.Json nuget package, the same nuget package and file dependency should also be referenced in the application project.

In the `SampleComponent.csproj` file, add the nuget package reference:

XML

```
<PackageReference Include="Newtonsoft.Json">
<Version>13.0.1</Version>
</PackageReference>
```

In the *CppToCSharpWinRT* project, locate the `packages.config` file and add the appropriate nuget reference. This will install the nuget package into the solution's package folder.

In `packages.config`, add the same nuget package reference:

XML

```
<package id="Newtonsoft.Json" version="13.0.1" targetFramework="native"
developmentDependency="true" />
```

Then add the following to the application project file to reference the appropriate file dependency from the solution's package folder. For example, in **CppToCSharpWinRT.vcxproj** add the following:

XML

```
<ItemGroup>
  <None
    Include="..\packages\Newtonsoft.Json.13.0.1\lib\netstandard2.0\Newtonsoft.Json.dll">
      <Link>%(Filename)%(Extension)</Link>
      <DeploymentContent>true</DeploymentContent>
    </None>
  </ItemGroup>
```

Related topics

- [Windows Runtime components with C# and Visual Basic](#)
- [Windows Runtime components with C++/WinRT](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Visual Studio native debug visualization (natvis) for C++/WinRT

Article • 10/20/2022

The [C++/WinRT Visual Studio Extension \(VSIX\)](#) gives you Visual Studio native debug visualization (natvis) of C++/WinRT projected types. This provides you an experience similar to C# debugging.

ⓘ Note

For more info about the C++/WinRT Visual Studio Extension (VSIX), see [Visual Studio support for C++/WinRT, and the VSIX](#).

Enabling natvis

Natvis is automatically on for a debug build because **WINRT_NATVIS** is defined when the **_DEBUG** symbol is defined.

Here's how to opt into it for a release build.

- Compile your code with the symbol **WINRT_NATVIS** defined. Doing so exports a **WINRT_abi_val** function, which provides the entry point for the debug visualizer to evaluate property values in the target process.
- Generate a full PDB. This is because the debug visualizer uses the Visual Studio C++ Expression Evaluator, which in turn requires symbolic definitions for displayed property types.
- A visualized type must report a runtime class or an interface defined in discoverable metadata. It does this via its implementation of [IInspectable::GetRuntimeClassName](#).

Given the above, the debug visualizer works best with Windows system types for which metadata can be found in the `C:\Windows\System32\WinMetadata` folder. However, it can also support user-defined types and remote debugging, provided that you properly locate `.winmd` files.

Using custom metadata

The debug visualizer looks for user-defined metadata (`.winmd` files) alongside the process `.exe`. It uses an algorithm similar to that of [RoGetMetaDataFile](#), probing for successive substrings of the fully-qualified typename. For example, if the type being visualized is **Contoso.Controls.Widget**, then the visualizer looks, in sequence, for:

- Contoso.Controls.Widget.winmd
- Contoso.Controls.winmd
- Contoso.winmd

Remote debugging with custom metadata

When remote debugging, the process `.exe` isn't local, so the search for custom metadata (mentioned in the previous section) fails. In that case, the visualizer falls back to a local cache folder (`%TEMP%`) for a suitable `.winmd` file. If it finds one, then it records the size and date of the file, and then searches the remote debugging target for the same `.winmd` alongside the binary. If necessary, the remote file is downloaded, updating the local cache. This strategy ensures that the locally cached `.winmd` is always up to date, as well as providing a means for manually caching a `.winmd` if it can't be found remotely (for example, if F5 deployment didn't put it there).

For an example of the caching behavior, see the [Troubleshooting](#) section below.

Troubleshooting

The debug visualizer uses the Visual Studio C++ Expression Evaluator to invoke the exported **WINRT_abi_val** function to obtain property values. Normally, the visualizer can catch unhandled exceptions, and degrade gracefully, displaying "<Object uninitialized or information unavailable>" in Visual Studio **Watch** windows.

That's useful when the visualizer tries to evaluate a local variable outside of its lifetime scope (for example, before construction). In some contexts, such as unit tests, an unhandled exception filter is installed. This can cause the process to terminate when the C++ expression evaluator faults. To prevent faulting, the visualizer makes several [VirtualQuery](#) calls in **WINRT_abi_val**.

Diagnostics

If a property isn't being displayed correctly, then turn on verbose natvis diagnostics in Visual Studio (**Tools > Options > Debugging > Output Window > Natvis diagnostic messages**), and then observe the **Output** window for natvis errors.

The following excerpt shows several attempts to probe for a `.winmd` file, followed by a download from the remote target to the local cache folder, and then a load of that `.winmd` file.

Console

```
Natvis C++/WinRT: Looking for
C:\Users\...\AppData\Local\DevelopmentFiles\ffcddd4f-cfc0-44cb-b736-
0b2d026def77VS.Debug_x64...\Consoso.Controls.Widget.winmd
Natvis C++/WinRT: Looking for
C:\Users\...\AppData\Local\DevelopmentFiles\ffcddd4f-cfc0-44cb-b736-
0b2d026def77VS.Debug_x64...\Consoso.Controls.winmd
Natvis C++/WinRT: Downloading
C:\Users\...\AppData\Local\DevelopmentFiles\ffcddd4f-cfc0-44cb-b736-
0b2d026def77VS.Debug_x64...\Consoso.Controls.winmd
Natvis C++/WinRT: Loaded
C:\Users\...\AppData\Local\Temp\Consoso.Controls.winmd
```

If the visualizer fails to find a `.winmd` file, then this error is generated:

Console

```
Natvis C++/WinRT: Could not find metadata for Consoso.Controls.Widget
```

There are a number of other error scenarios that all produce diagnostics.

If metadata is available, then the output diagnostics will show many calls like this:

Console

```
Natvis C++/WinRT: WINRT_abi_val(*(::IUnknown**)0x32dd4ffc18, L"{96369F54-
8EB6-48F0-ABCE-C1B211E627C3}", 0).s,sh
Natvis C++/WinRT: WINRT_abi_val(*(::IUnknown**)0x32dd4ffc18, L"{AF86E2E0-
B12D-4C6A-9C5A-D7AA65101E90}", -2).s,sh
```

The first is a call to [IStringable.ToString](#) to obtain the string representation of a complex type (the unexpanded display value).

The second is a call to [IInspectable::GetRuntimeClassName](#), in order to reflect on the type's properties.

Subsequent `WINRT_abi_val` calls are property evaluations for each interface discovered on the type.

Invoking `WINRT_abi_val`

You can use the Visual Studio **Immediate/Command** windows to directly invoke **WINRT_abi_val** for troubleshooting.

For example, given a projected variable *stringable*, you can evaluate its [IStringable.ToString](#) as:

Console

```
>? WINRT_abi_val((::IUnknown*)&stringable, L"{96369F54-8EB6-48F0-ABCE-  
C1B211E627C3}", 0).s,sh  
L"string"
```

Related topics

- [Visual Studio support for C++/WinRT, and the VSIX](#)

C++/WinRT configuration macros

Article • 12/20/2024

This topic describes the C++/WinRT configuration macros. Unless otherwise noted, these rules apply to all of the C++/WinRT configuration macros:

- All files that are linked together to form a single module (`.exe` or `.dll`) must have identical macro settings. That includes static libraries.
- All macro settings must be complete before including any C++/WinRT header file.
- You may not change any macro setting after including any C++/WinRT header file.

WINRT_LEAN_AND_MEAN

If defined, disables these rarely-used features (in order to reduce compile times):

- The ability to implement exclusive interfaces outside the component.
- `std::hash` specializations for interface and runtime class smart pointers.
- Support for directly outputting an `hstring` or `IStringable` to a C++ stream, since version 2.0.221101.3.

You're allowed to combine files with different settings for `WINRT_LEAN_AND_MEAN`.

Files that don't define `WINRT_LEAN_AND_MEAN` gain access to the rarely-used features.

WINRT_NO_MODULE_LOCK

If defined, disables object counts for the current module. The module never unloads from the process. Defining this macro is customary for executables (which can never unload), or for `.dll`s that you intend to leave pinned. May not be combined with `WINRT_CUSTOM_MODULE_LOCK`.

WINRT_CUSTOM_MODULE_LOCK

If defined, allows you to provide your own implementation of `winrt::get_module_lock`. May not be combined with `WINRT_NO_MODULE_LOCK`.

Your custom implementation of `winrt::get_module_lock` must support the following operations:

- `++winrt::get_module_lock()`: Increment the reference count on the module lock.

- `--winrt::get_module_lock()`: Decrement the reference count on the module lock.
- `if (winrt::get_module_lock())`: Check whether the reference count is nonzero.
(Needed if you're building a DLL.)

WINRT_ASSERT, WINRT_VERIFY

These macros allow you to customize assertion handling. **WINRT_ASSERT** doesn't require the argument to be evaluated. **WINRT_VERIFY** requires that the argument be evaluated, even in non-debug builds.

If you don't customize these macros, and **_DEBUG** is defined, then C++/WinRT makes them equivalent to **_ASSERT**.

If you don't customize these macros, and **_DEBUG** is not defined, then C++/WinRT defines **WINRT_ASSERT** to discard the expression unevaluated, and defines **WINRT_VERIFY** to discard the expression after evaluating it.

WINRT_NO_MAKE_DETECTION

If defined, disables the default C++/WinRT diagnostic that detects that you mistakenly constructed an implementation class without using [winrt::make](#).

We strongly recommend that you don't define this symbol, because doing so masks a common source of programming errors.

WINRT_NO_SOURCE_LOCATION

If defined, disables the inclusion of source file and line number information (and in debug builds, function information), when originating errors.

This additional information is not used by C++/WinRT, but it is made available to other libraries which wish to interoperate with C++/WinRT exceptions, such as the [Windows Implementation Library](#).

By default, the information is included when compiled in C++20 mode or higher. You may wish to suppress this information to reduce binary size.

WINRT_DIAGNOSTICS

If defined, enables internal statistics to track various operations:

- The number of times each interface was queried.
- The number of times each factory was requested (and whether the factory is agile).

WINRT_NATVIS

If defined, includes helper functions to assist in native debug visualizations in Visual Studio. The code isn't used at runtime; it exists only for debugging.

If you don't customize this macro, then visualization support functions are enabled if `_DEBUG` is defined. For more details, see [Visual Studio native debug visualization \(natvis\) for C++/WinRT](#).

You're allowed to combine files with different settings for `WINRT_NATVIS`.

If any file is compiled with `WINRT_NATVIS` support, then the resulting module will have native debug visualizations enabled.

WINRT_EXPORT, WINRT_FAST_ABI_SIZE

Don't use these macros.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

C++/WinRT naming conventions

Article • 10/20/2022

C++/WinRT has established the following naming conventions:

- The **winrt::impl** namespace is reserved for C++/WinRT, and you shouldn't use it in your application.
- In the **winrt** namespace, names that begin with a lowercase letter belong to C++/WinRT, but you may use them in your application. The documentation calls out those names that you can overload or specialize. For example, your application is permitted to specialize the [winrt::is_guid_of](#) function template.
- In sub-namespaces of the **winrt** namespace (except for **winrt::impl**), names that begin with an uppercase letter are available to your application.
- In all namespaces, names beginning with **WINRT_IMPL_** are reserved for C++/WinRT, and you shouldn't use them in your application.
- In all namespaces, names beginning with **WINRT_** (except those that begin with **WINRT_IMPL_**) are reserved for C++/WinRT. You may use them, and the documentation calls out those names that may be defined by your application, such as **WINRT_LEAN_AND_MEAN**.

It's common for applications to perform namespace composition, and import sub-namespaces of the **winrt** root namespace into the **winrt** root namespace:

C++

```
namespace winrt
{
    using namespace winrt::Windows::Foundation;
}
```

Therefore, your application should adhere to the naming conventions above in sub-namespaces of the **winrt** namespace.

Here's a summary.

Namespace	Name	Apps may define	Apps may use
winrt::impl	Any	No	No
winrt and sub-namespaces (except impl)	Starts with lowercase letter	No	Yes

Namespace	Name	Apps may define	Apps may use
winrt and sub-namespaces (except impl)	Starts with uppercase letter	Yes	Yes
Any	WINRT_IMPL_*	No	No
Any	WINRT_* (except WINRT_IMPL_*)	Case-by-case	Yes

Value categories, and references to them

Article • 12/31/2022

This topic introduces and describes the various categories of values (and references to values) that exist in C++:

- glvalue
- lvalue
- xvalue
- prvalue
- rvalue

You'll doubtless have heard of *lvalues* and *rvalues*. But you might not think of them in the terms that this topic presents.

Every expression in C++ yields a value that belongs to one of the five categories listed above. There are aspects of the C++ language—its facilities and rules—that demand a proper understanding of these value categories, as well as references to them. These aspects include taking the address of a value, copying a value, moving a value, and forwarding a value on to another function. This topic doesn't go into all of those aspects in depth, but it provides foundational information for a solid understanding of them.

The info in this topic is framed in terms of Stroustrup's analysis of value categories by the two independent properties of identity and movability [Stroustrup, 2013].

An lvalue has identity

What does it mean for a value to have *identity*? If you have (or you can take) the memory address of a value, and use it safely, then the value has identity. That way, you can do more than compare the contents of values—you can compare or distinguish them by identity.

An *lvalue* has identity. It's now a matter of only historical interest that the "l" in "lvalue" is an abbreviation of "left" (as in, the left-hand-side of an assignment). In C++, an lvalue can appear on the left *or* on the right of an assignment. The "l" in "lvalue", then, doesn't actually help you to comprehend nor define what they are. You need only to understand that what we call an lvalue is a value that has identity.

Examples of expressions that are lvalues include: a named variable or constant; or a function that returns a reference. Examples of expressions that are *not* lvalues include: a

temporary; or a function that returns by value.

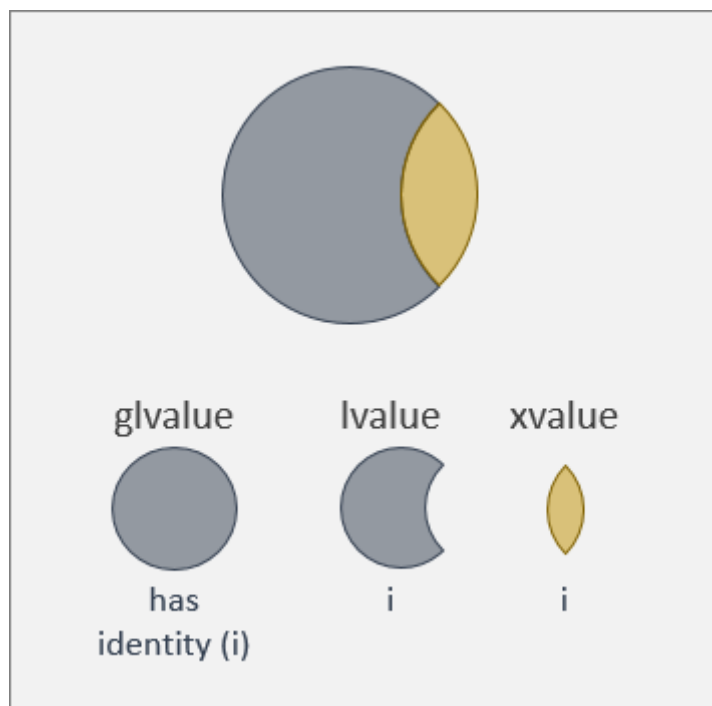
C++/WinRT

```
int& get_by_ref() { ... }
int get_by_val() { ... }

int main()
{
    std::vector<byte> vec{ 99, 98, 97 };
    std::vector<byte>* addr1{ &vec }; // ok: vec is an lvalue.
    int* addr2{ &get_by_ref() }; // ok: get_by_ref() is an lvalue.

    int* addr3{ &(get_by_ref() + 1) }; // Error: get_by_ref() + 1 is not an
    lvalue.
    int* addr4{ &get_by_val() }; // Error: get_by_val() is not an lvalue.
}
```

Now, while it's a true statement that lvalues have identity, that's also true of xvalues. We'll go into exactly what an *xvalue* is later in this topic. For now, just be aware that there's a value category called *glvalue* (for "generalized lvalue"). The set of glvalues is the superset of both lvalues (also known as *classical lvalues*) and xvalues. So, while "an lvalue has identity" is true, the complete set of things that have identity is the set of glvalues, as shown in this illustration.



An rvalue is movable; an lvalue is not

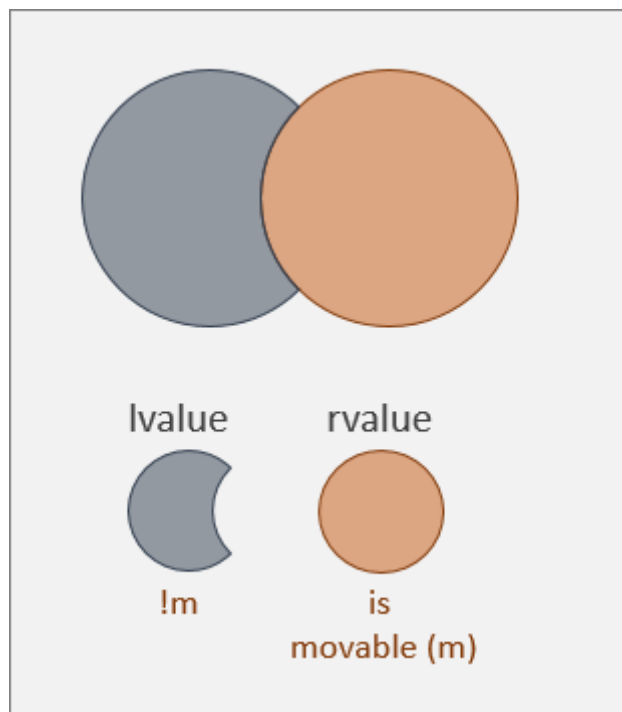
But there are values that are not glvalues. In other words, there are values that you *can't* obtain a memory address for (or you can't rely on it to be valid). We saw some such

values in the code example above.

Not having a reliable memory address sounds like a disadvantage. But in fact the advantage of a value like that is that you can *move* it (which is generally cheap), rather than copy it (which is generally expensive). Moving a value means that it's no longer in the place where it used to be. So trying to access it in the place where it used to be is something to be avoided. A discussion of when and *how* to move a value is out of scope for this topic. For this topic, we just need to know that a movable value is known as an *rvalue* (or *classical rvalue*).

The "r" in "rvalue" is an abbreviation of "right" (as in, the right-hand-side of an assignment). But you can use rvalues, and references to rvalues, outside of assignments. The "r" in "rvalue", then, is not the thing to focus on. You need only to understand that what we call an rvalue is a value that is movable.

An lvalue, conversely, is not movable, as shown in this illustration. If an lvalue were to move, then that would contradict the very definition of *lvalue*. And it would be an unexpected problem for code that very reasonably expected to be able to continue to access the lvalue.



So you can't move an lvalue. But there *is* a kind of glvalue (the set of things with identity) that you can move—if you know what you're doing (including being careful not to access it after the move)—and that's the xvalue. We'll revisit that idea one more time later in this topic when we look at the complete picture of value categories.

Rvalue references, and reference-binding rules

This section introduces the syntax for a reference to an rvalue. We'll have to wait for another topic to go into a substantial treatment of moving and forwarding, but suffice to say that rvalue references are a necessary piece of the solution of those problems. Before we look at rvalue references, though, we first need to be clearer about `T&`—the thing we've formerly been calling just "a reference". It's really "an lvalue (non-const) reference", which refers to a value to which the user of the reference can write.

C++/WinRT

```
template<typename T> T& get_by_lvalue_ref() { ... } // Get by lvalue (non-const) reference.
template<typename T> void set_by_lvalue_ref(T&) { ... } // Set by lvalue (non-const) reference.
```

An lvalue reference can bind to an lvalue, but not to an rvalue.

Then there are lvalue const references (`T const&`), which refer to objects to which the user of the reference *can't* write (for example, a constant).

C++/WinRT

```
template<typename T> T const& get_by_lvalue_cref() { ... } // Get by lvalue const reference.
template<typename T> void set_by_lvalue_cref(T const&) { ... } // Set by lvalue const reference.
```

An lvalue const reference can bind to an lvalue or to an rvalue.

The syntax for a reference to an rvalue of type `T` is written as `T&&`. An rvalue reference refers to a movable value—a value whose contents we don't need to preserve after we've used it (for example, a temporary). Since the whole point is to move from (thereby modifying) the value bound to an rvalue reference, `const` and `volatile` qualifiers (also known as cv-qualifiers) don't apply to rvalue references.

C++/WinRT

```
template<typename T> T&& get_by_rvalue_ref() { ... } // Get by rvalue reference.
struct A { A(A&& other) { ... } }; // A move constructor takes an rvalue reference.
```

An rvalue reference binds to an rvalue. In fact, in terms of overload resolution, an rvalue *prefers* to be bound to an rvalue reference than to an lvalue const reference. But an rvalue reference can't bind to an lvalue because, as we've said, an rvalue reference refers

to a value whose contents it's assumed we don't need to preserve (say, the parameter for a move constructor).

You can also pass an rvalue where a by-value argument is expected, via copy construction (or via move construction if the rvalue is an xvalue).

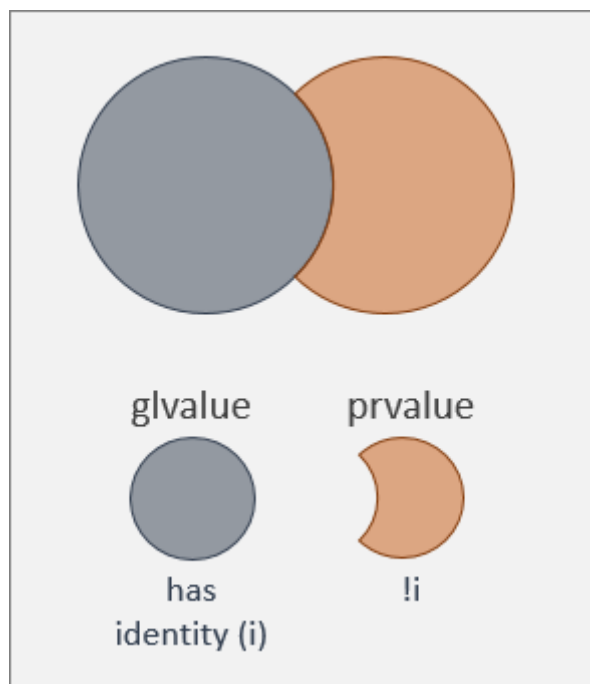
A glvalue has identity; a prvalue does not

At this stage, we know what has identity. And we know what's movable and what isn't. But we haven't yet named the set of values that *don't* have identity. That set is known as the *prvalue*, or *pure rvalue*.

C++/WinRT

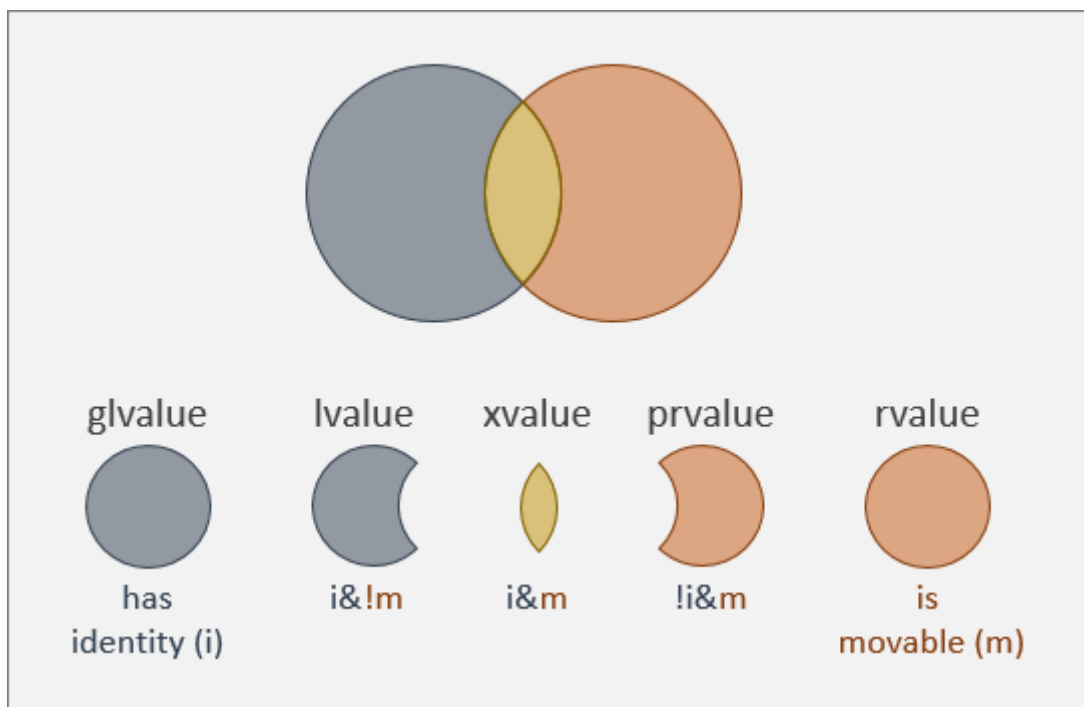
```
int& get_by_ref() { ... }
int get_by_val() { ... }

int main()
{
    int* addr3{ &(get_by_ref() + 1) }; // Error: get_by_ref() + 1 is a
    prvalue.
    int* addr4{ &get_by_val() }; // Error: get_by_val() is a prvalue.
}
```



The complete picture of value categories

It only remains to combine the info and illustrations above into a single, big picture.



glvalue (i)

A glvalue (generalized lvalue) has identity. We'll use "i" as a shorthand for "has identity".

lvalue (i&!m)

An lvalue (a kind of glvalue) has identity, but isn't movable. These are typically read-write values that you pass around by reference or by const reference, or by value if copying is cheap. An lvalue can't be bound to an rvalue reference.

xvalue (i&m)

An xvalue (a kind of glvalue, but also a kind of rvalue) has identity, and is also movable. This might be an erstwhile lvalue that you've decided to move because copying is expensive, and you'll be careful not to access it afterward. Here's how you can turn an lvalue into an xvalue.

C++/WinRT

```
struct A { ... };  
A a; // a is an lvalue...  
static_cast<A&&>(a); // ...but this expression is an xvalue.
```

In the code example above, we haven't moved anything yet. We've merely created an xvalue by casting an lvalue to an unnamed rvalue reference. It can still be identified by its lvalue name; but, as an xvalue, it is now *capable* of being moved. The reasons for

moving it, and what moving actually looks like, will have to wait for another topic. But you can think of the "x" in "xvalue" as meaning "expert-only" if that helps. By casting an lvalue into an xvalue (a kind of rvalue, remember), the value then becomes capable of being bound to an rvalue reference.

Here are two other examples of xvalues—calling a function that returns an unnamed rvalue reference, and accessing a member of an xvalue.

C++/WinRT

```
struct A { int m; };  
A&& f();  
f(); // This expression is an xvalue...  
f().m; // ...and so is this.
```

prvalue (!l&m)

A prvalue (pure rvalue; a kind of rvalue) doesn't have identity, but is movable. These are typically temporaries, or the result of calling a function that returns by value, or the result of evaluating any other expression that's not a glvalue.

rvalue (m)

An rvalue is movable. We'll use "m" as a shorthand for "is movable".

An rvalue *reference* always refers to an rvalue (a value whose contents it's assumed we don't need to preserve).

But, is an rvalue reference itself an rvalue? An *unnamed* rvalue reference (like the ones shown in the xvalue code examples above) is an xvalue so, yes, it's an rvalue. It prefers to be bound to an rvalue reference function parameter, such as that of a move constructor. Conversely (and perhaps counter-intuitively), if an rvalue reference has a name, then the expression consisting of that name is an lvalue. So it *can't* be bound to an rvalue reference parameter. But it's easy to make it do so—just cast it to an unnamed rvalue reference (an xvalue) again.

C++/WinRT

```
void foo(A&) { ... }  
void foo(A&&) { ... }  
void bar(A&& a) // a is a named rvalue reference; so it's an lvalue.  
{  
    foo(a); // Calls foo(A&).  
    foo(static_cast<A&&>(a)); // Calls foo(A&&).  
}
```